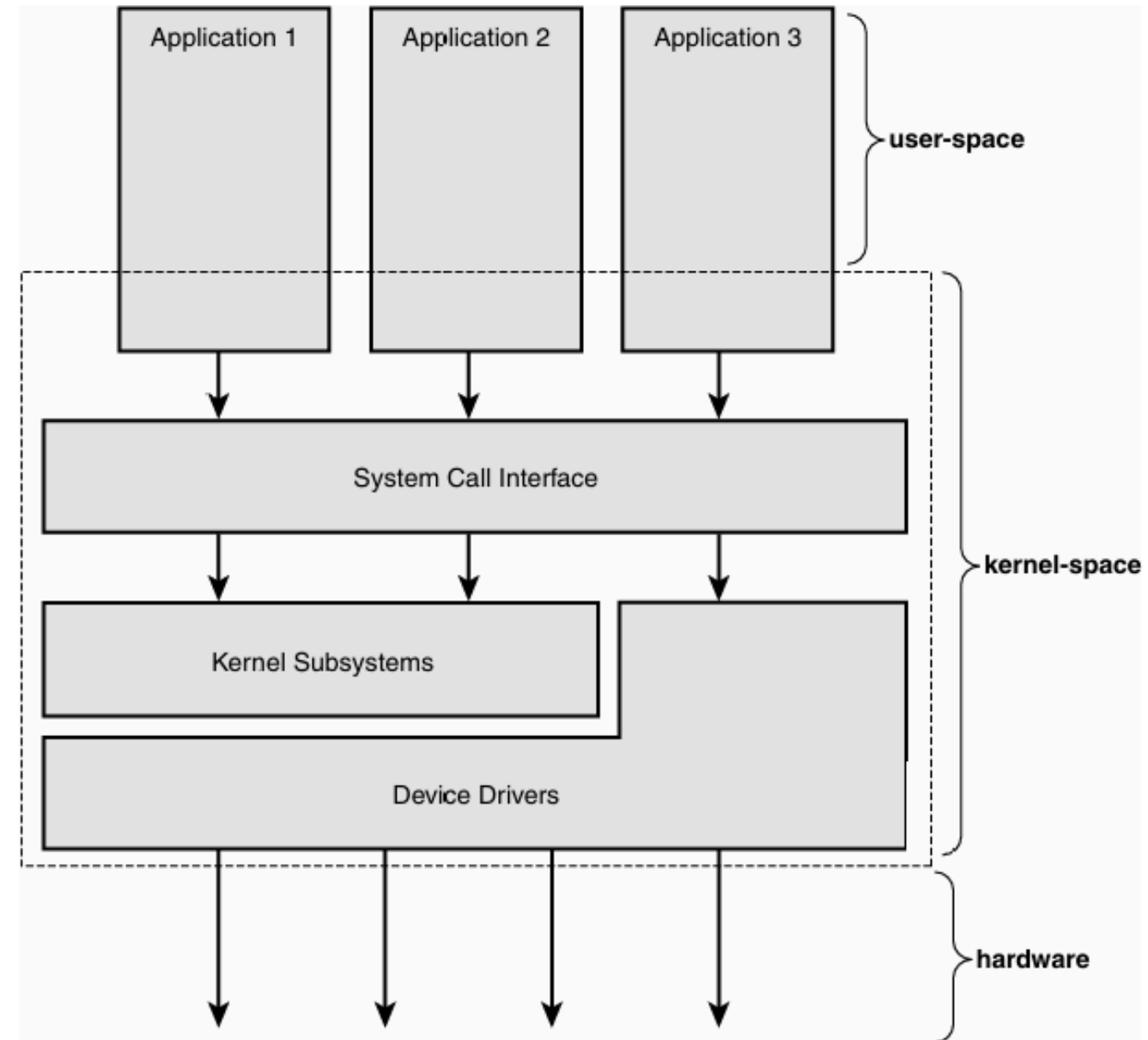


Linux Kernel Developing

Introduction

Overview of Operating Systems and Kernels



Linux Source Code and Linux Directory Tree

Source Code available at:

<https://github.com/torvalds/linux/tree/master>

Directory	Description
arch/	Architecture-specific source code
block/	Block I/O subsystem
certs/	Certificates and kernel key handling
crypto/	Cryptographic subsystem
Documentation/	Kernel documentation
drivers/	Device drivers
fs/	Virtual File System (VFS) and individual filesystems
include/	Kernel header files
init/	Kernel boot and initialization code
io_uring/	Asynchronous I/O subsystem
ipc/	Interprocess communication code
kernel/	Core kernel functionality, such as scheduling and timers
lib/	Common helper and utility routines
mm/	Memory management subsystem and virtual memory
net/	Networking subsystem
rust/	Rust support for kernel development
samples/	Sample and demonstration kernel code
scripts/	Scripts used to build and configure the kernel
security/	Kernel security infrastructure and Linux Security Modules
sound/	Audio subsystem
tools/	Development, testing, tracing, and performance tools
usr/	Support for the built-in initial RAM filesystem (initramfs)
virt/	Virtualization infrastructure

C in kernel development

Language & Toolchain

- Linux is primarily written in C.
- Current kernel builds typically use **-std=gnu11** (GNU dialect of ISO C11).
- GCC and Clang/LLVM are supported.

GNU C Extensions You Will See

- **typeof / __typeof__**
- Statement expressions: **{ ... }**
- Attributes: **__attribute__((...))**
- Inline functions, built-ins, and inline assembly.

Examples

```
/* typeof + statement expression */
#define MIN(a,b) ({
    typeof(a) _a = (a);
    typeof(b) _b = (b);
    _a < _b ? _a : _b;
})

/* compiler attribute */
__attribute__((noreturn))
void panic(const char *fmt, ...);
```

Kernel C ≠ User-space C

- **No normal libc/runtime**
Kernel uses its own APIs (e.g., printk, kmalloc).
- **No memory isolation inside the kernel**
No SIGSEGV will be thrown if the kernel access the wrong memory.
- **No (easy) use Floating point**
FP/SIMD state is not casually used; special critical-section APIs exist when required.

Reference Books and Documents.

Primary Source:

- <https://docs.kernel.org/>
- Documents in the repository: <https://github.com/torvalds/linux/blob/master/README>

Books:

- Linux Kernel Development, 3rd edition by Robert Love (Very Old 2010)
- Linux Kernel Programming, 2nd Edition by Kaiwan N. Billimoria